서울 지하철 출발 시각 추천 서비스 최종 기획서

Nov 10, 2026 · @gyu

1. 개요

출퇴근 시간대의 지하철 지연을 머신러닝으로 예측해, 목표 시각에 안정적으로 도착할 수 있는 출발 시각과 열차를 추천하는 서비스를 4주 동안 2인으로 만든다. 대상은 서울 지하철이며, 도보 시간은 제외하고 역 안에서의 이동(탑승, 환승)만 계산한다.

문제. 기존 지하철 앱은 시간표 기준의 경로와 도착 예정 시각만 보여준다. 실제로는 지연이 생기고 환승할 때 연결 열차를 놓치기도 해서, 같은 열차를 타도 도착 시각이 매일 달라진다. 그래서 사람들은 감으로 여유를 두고 출발한다.

목표.

- 사용자: "몇 시 몇 분 열차를 타야 하는지"를 근거 있는 숫자로 제공한다.
- 팀: Kafka 실시간 수집, Flink 스트림 처리, Spark 배치 분석, Airflow 자동화, ML 지연 예측, AI API 활용까지 이어지는 서비스를 직접 설계하고 운영한다.

2. 서비스 기능

사용자는 노선도에서 역을 고르거나 문장으로 입력하고, 서비스는 출발역에서 탈 열차를 시각과 열차 번호로 알려준다. "출발 시각"은 출발역에서 열차에 타는 시각이다.

사용자 흐름

1. 노선도에서 출발역을 클릭하거나 "내일 9시까지 강남역"처럼 문장으로 입력한다.
2. 필요하면 경유역(경로가 반드시 지나야 하는 역)을 추가한다.
3. 도착역과 도착해야 하는 시각을 정한다.
4. 결과를 확인한다.

결과 화면

- 노선도 위 경로 하이라이트(노선, 환승역)
- 안정 출발 시각과 마지노선 출발 시각(각각 열차 번호 포함)
- 예상 도착 시각의 범위(빠를 때~늦을 때)와 구간별 예상 소요 시간
- 환승별 연결 성공 확률
- 경로상 지금 지연이 생긴 구간 경고와 추천 근거 한 줄 설명
- 노선도 위 운행 중인 열차 위치

두 가지 출발 시각

구분	의미	근거
안정 출발 시각	이 열차를 타면 20번 중 19번은 제시간에 도착하는 (P95) 열차 중 가장 늦은 열차의 출발 시각	ML 지연 분위 예측 (과거 이력, 배치)
마지노선 출발 시각	지금 운행 중인 열차의 실제 위치와 지연을 반영했을 때 목표 시각 안에 도착할 수 있는 마지막 열차	실시간 상태 (스트림)

두 시각의 근거가 배치와 스트림으로 나뉘어 있어서, 두 파이프라인을 모두 만들어야 서비스가 성립한다. P95 기준은 검증 결과를 보고 조정한다.

3. 계산 방식

경로를 탑승 구간과 환승 구간으로 나눠 구간마다 소요 시간을 구하고 이어 붙인다. 전체 소요 시간은 출발역 승차부터 도착역 하차까지다.

```
T_{\text{total}} = \sum_{\text{탑승 구간}} (\text{시간표 소요} + \text{지연}) + \sum_{\text{환승}} (\text{환승 이동} + \text{다음 열차 대기})
```

- **탑승 구간**: 역과 역 사이 이동 시간이다. 시간표 기준값에 지연을 더한다. 실시간 계산은 해당 열차의 현재 지연을, 안정 출발 시각 계산은 ML이 예측한 지연 분위값(P95)을 쓴다.
- **환승 구간**: 환승 통로 이동 시간과 다음 열차 대기 시간이다. 앞 열차가 늦으면 연결 열차를 놓칠 수 있어서, 연결 성공 확률을 따로 예측한다(4장). 환승 이동 시간은 공개된 환승 소요 시간을 기본값으로 쓴다.
- **경로 탐색**: 역을 노드, 역간 이동과 환승을 엣지로 하는 그래프에서 출발역 → 경유역 → … → 도착역을 구간별 다익스트라로 찾아 이어 붙인다.
- **출발 시각 산출**: 목표 시각 이전에 출발역을 떠나는 열차들을 후보로 놓고 각 열차의 도착 시각을 계산한다. 목표를 만족하는 가장 늦은 열차를, P95 기준이면 안정 출발 시각으로, 실시간 기준이면 마지노선 출발 시각으로 낸다.

4. 머신러닝 모델

ML은 열차·구간별 지연의 분포(P50, P95)를 예측하고, 그 결과로 "이 열차를 타면 95% 확률로 제시간에 도착한다"를 계산한다. 30분 시간대별 단순 통계를 기준선으로 두고, 모델이 같은 정시율을 지키면서 출발을 얼마나 늦출 수 있는지로 가치를 증명한다.

항목	내용
학습 단위	열차 1대가 역 1곳에 도착한 이벤트 (열차번호 × 역)
정답	지연 = 실제 도착 시각 − 시간표 도착 시각

항목	내용
입력	호선·방향·역, 시각(분), 요일, 공휴일 여부, 앞차와의 간격, 앞차 지연, 같은 열차의 직전 역 지연, 역별 시간대 승하차 인원(월 통계), 강수량(선택)
모델	LightGBM 분위 회귀 (P50, P95)
기준선	평일/주말 × 30분 시간대별 P95 통계
추가 모델	환승 성공 확률 분류: 앞 열차 지연 예측값, 환승 이동 시간, 연결 열차 간격 → 연결 성공 여부

두 상황을 나눠 학습한다. 미리 계획할 때(전날 밤, 출발 1시간 전)는 현재 지연을 알 수 없으므로, 요일·시간대·날씨 예보·과거 패턴만 쓰는 모델로 안정 출발 시각을 낸다. 출발 직전에는 현재 노선 지연과 앞차 상태까지 넣어 마지노선을 보정한다. 나누지 않으면 학습 때만 쓸 수 있는 정보를 쓰는 데이터 누수가 생긴다.

평가.

- 커버리지: 실제 지연이 예측 P95 이하인 비율이 95% 근처인가
- Pinball loss: 기준선 대비 분위 예측 오차
- 서비스 지표: 같은 95% 정시 도착률을 지키면서 기준선보다 출발을 몇 분 늦출 수 있었나
- 데이터 분할은 날짜 기준(마지막 1주 테스트)으로 한다. 무작위로 섞으면 같은 날 같은 열차 정보가 학습·테스트에 함께 들어가 성능이 부풀려진다.

EDA. 1주차 수집 데이터로 구간별 지연 분포, 시간대별 환승 실패율, 노선별 정시성을 먼저 본다. 지하철 지연이 대부분 0~1분이면 환승 성공 확률 모델에 무게를 더 둔다.

5. AI 활용

LLM API를 입력과 설명 두 곳에 쓴다. 계산 자체는 ML과 규칙이 하고, LLM은 사람의 말을 계산 입력으로 바꾸고 계산 결과를 사람의 말로 바꾼다.

- **자연어 입력**: "내일 9시까지 강남역, 신도림 거쳐서" → 출발역·경유역·도착역·목표 시각 JSON으로 변환한다. 역명은 역 목록과 대조해 검증하고, 애매하면 다시 묻는다.
- **추천 근거 설명**: "지금 2호선 외선 4분 지연이라 안정 출발을 8:12에서 8:07로 당겼어요"처럼 계산 결과와 현재 지연을 한 줄로 요약한다. 숫자는 계산 결과만 쓰고 LLM이 만들어내지 않게 한다.

6. 데이터

메인 수집원은 노선 단위로 호출하는 실시간 열차 위치정보이고, 나머지는 지연 계산 기준값, 그래프 구성, ML 특성용이다. 과거 위치 데이터는 제공되지 않으므로 이력은 수집을 시작한 날부터 생긴다.

데이터	출처	용도	수집	상태 (2026-11-10)
실시간 열차 위치정보	서울 열린데이터 광장	열차별 궤적, 역 도착 이벤트, 지연 계산	실시간, 노선 단위	운영 중, 오늘 수집 시작
실시간 도착정보(일괄)	서울 열린데이터 광장	도착 시각 샘플 검증	보조	2026년 6월 일시 중단 후 7월 정상화
열차시간표	공공데이터포털 (서울교통공사)	지연 기준값, 후보 열차	배치	열차번호·출도착 시각 제공, 개발계정 하루 10,000회
역간 소요 시간	공공데이터포털	그래프 엣지 가중치	배치	1주차 확인
환승 소요 시간	공공데이터포털	환승 엣지 가중치	배치	1주차 확인
역 목록·좌표	공공데이터포털	그래프 노드, 노선도 UI	배치	1주차 확인
역별 시간대별 승하차 인원	서울 열린데이터 광장	ML 특성(혼잡도)	월 1회	매월 5일 전월분 갱신

지연 산출. 실시간 API는 "지연 몇 분"을 직접 주지 않으므로 직접 계산한다.

1. 열차 번호별로 위치 수신 기록을 모아 궤적을 만든다.
2. 궤적에서 각 역에 실제로 도착한 시각을 뽑는다(같은 위치 반복 수신, 수신 누락 처리, 데이터 생성 시각 기준).
3. 같은 열차 번호의 시간표 도착 시각과 비교해 역별 지연을 구한다.

7. 수집 범위와 출퇴근 시간대

출퇴근 핵심 시간은 서울시 기준으로 출근 07~09시, 퇴근 18~20시다. 수집은 앞뒤 30분 여유를 둬 하루 6시간만 집중한다. (서울시)

구분	핵심 시간	수집 시간	길이
출근	07:00~09:00	06:30~09:30	3시간
퇴근	18:00~20:00	17:30~20:30	3시간

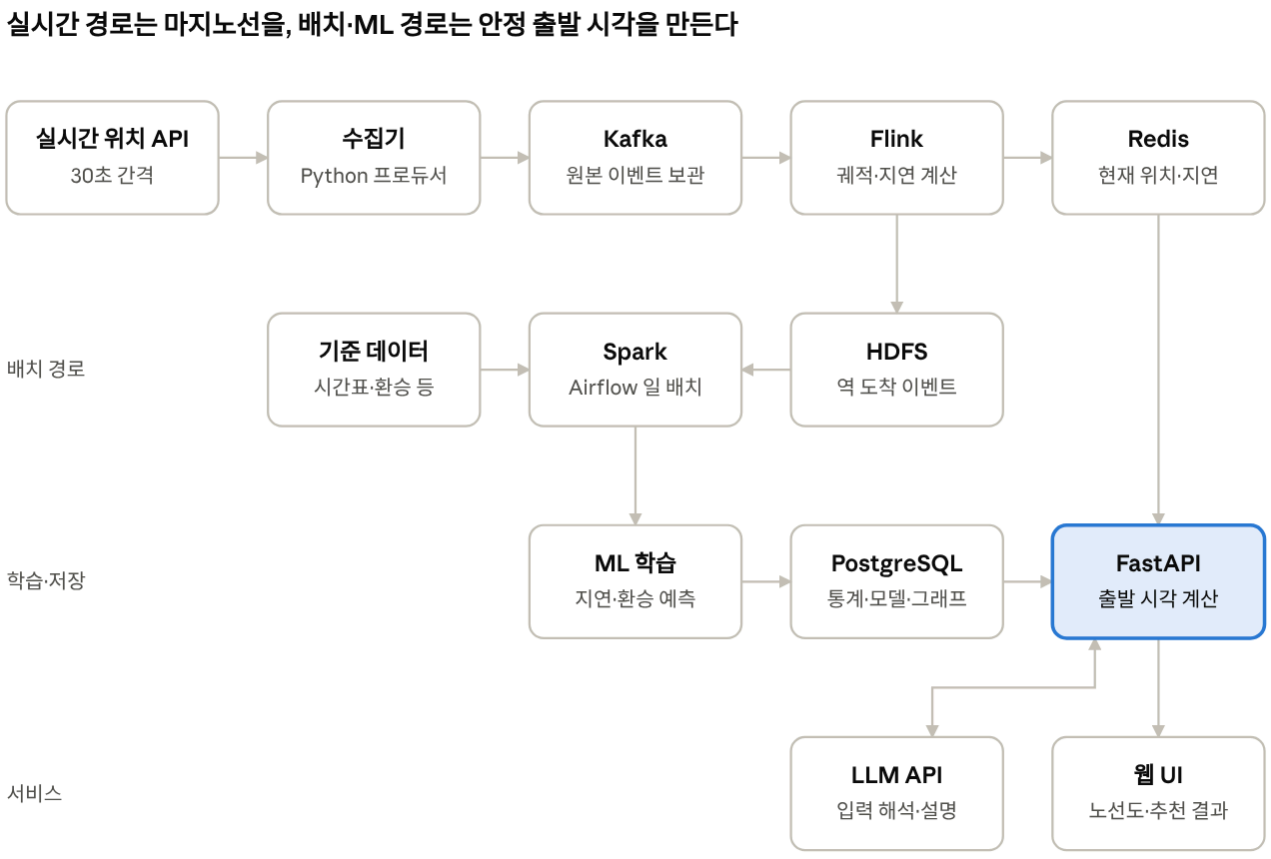
2014년 교통카드 분석에서 출근 2시간에 하루 이용객의 20.2%, 퇴근 2시간에 18.3%가 몰렸고, 가장 붐빈 10분은 08:10~08:19였다. (내 손안에 서울) 오래된 자료라 실제 피크는 승하차 인원 최신 통계로 1주차에 다시 확인한다.

호출 예산. 실시간 지하철 API는 인증키당 하루 1,000건까지다. (열린데이터광장 이용안내) 위치 API는 노선 하나를 한 번 부르면 그 노선 열차 전체가 온다.

설정	하루 호출	한도 안
2호선 1개, 30초 간격, 6시간	720	예
노선 2개, 60초 간격, 6시간	720	예
노선 2개, 30초 간격, 6시간	1,440	아니오

30초 간격이 도착 시각 정밀도에 유리하므로 처음에는 2호선 하나를 30초로 모은다. 열린데이터광장 활용사례(갤러리)에 등록하면 제약 없이 쓸 수 있다고 안내돼 있어, 1주차에 등록해 노선을 늘린다. 주말도 같은 시간에 모아 평일/주말 비교에 쓴다.

8. 시스템 아키텍처



데이터 흐름 · 실시간, 배치·ML, 서비스

Flink가 계산한 현재 지연은 Redis로, 역 도착 이벤트는 HDFS(Parquet)로 갈라진다. Airflow가 매일 Spark 배치로 통계 기준선과 학습 데이터를 만들고, ML 모델과 통계는 PostgreSQL에 저장된다. 두 경로는 FastAPI에서 만나 출발 시각을 계산하고, LLM API가 입력을 해석하고 결과를 설명한다. Grafana는 consumer lag, 처리 지연, API 실패율을 감시하고, 개발 환경은 Docker Compose로 통일한다.

스트림은 Flink, 배치는 Spark로 나눠 가이드의 Kafka·Flink·Spark·Airflow 요소를 모두 쓴다.

9. MVP 범위

수집은 2호선부터 시작해 호출 제한이 풀리면 넓히고, 서비스 계산과 노선도는 일부 노선으로 한정한다.

포함

- 노선: 2호선 + 출퇴근 수요가 많은 서울교통공사 운영 노선 1~2개와 그 사이 환승역 (1주차 확정)
- 시간대: 평일·주말 출근 06:30~09:30, 퇴근 17:30~20:30
- 경유역: 0~2개
- ML: 지연 분위 회귀(P50, P95), 환승 성공 확률 분류, 30분 통계 기준선 비교
- AI: 자연어 입력, 추천 근거 한 줄 설명
- 노선도 UI: 역 클릭으로 출발·경유·도착 지정, 경로 하이라이트, 실시간 열차 위치

제외

- 도보 시간(집→역, 역→목적지)
- 버스 등 다른 교통수단, 9호선·코레일 구간처럼 서울교통공사 시간표 밖 노선
- 회원·알림 기능(시간이 남으면 추가)

노선도 제작 원칙. 기존 앱의 노선도 이미지는 저작권 문제가 있으므로, 공개된 역 좌표로 직접 그리거나 단순화한 노선도를 SVG로 만든다.

10. 역할 분담과 4주 일정

A는 수집·스트림·실시간, B는 배치·ML·서비스를 맡는다. 이력이 쌓여야 학습이 되므로 수집은 첫날(2026-11-10 퇴근 시간)부터 켠다.

주차	A (수집·스트림)	B (배치·ML·서비스)
0일차	수집기로 원본 파일 적재 시작	시간표·환승 데이터 위치 확인
1주	API 호출 제한 확인, Kafka 구축, 수집기 → Kafka 전환, 활용사례 등록	시간표 매칭률 확인, 역·노선 그래프, EDA
2주	Flink: 궤적 생성, 역 도착 이벤트, 지연 계산	Spark 배치 + Airflow: 30분 통계 기준선, 학습 데이터셋
3주	Redis 실시간 상태, 장애 복구 테스트, 마지노선 계산	ML 학습(분위 회귀, 환승 분류), 경로 탐색·출발 시각 API, LLM 입력·설명
4주	모니터링, 리플레이 부하 테스트, 실시간 열차 표시	노선도 UI 마무리, 정확도 검증(정시율·출발 늦춤 분)
공통	문서화, 발표 자료	문서화, 발표 자료

노선도 UI는 손이 많이 가므로 2~3주차에 여유가 있는 쪽이 먼저 시작한다. 매주 끝에 결과물을 연결해 보는 통합 점검을 한다.

1주차 체크리스트

- [x] ~~대상 노선 확정~~
- [ ] 시간표 열차번호와 실시간 열차번호 매칭률 확인
- [ ] 열린데이터광장 활용사례 등록(호출 제한 해제)
- [ ] Docker Compose 공통 개발 환경
- [ ] Kafka 토픽·메시지 스키마 합의

11. 검증과 성공 지표

성공 여부는 "추천한 열차를 탔다면 실제로 제시간에 도착했는가"로 판단한다. 쌓인 도착 이벤트로 추천 결과를 거꾸로 대조해 수치를 낸다.

분야	지표
서비스	안정 출발 시각을 따랐을 때 정시 도착률(목표 95%), 마지노선 예측 도착 시각과 실제의 오차
ML	P95 커버리지, Pinball loss, 기준선 대비 같은 정시율에서 출발을 늦출 수 있었던 시간(분), 환승 성공 분류 AUC
파이프라인	수집~처리 지연, Kafka consumer lag, API 호출 실패율, 리플레이 처리량
장애 대응	컨슈머·Flink 재시작 후 중복·누락 없이 복구되는지(체크포인트, watermark)
데이터 품질	수집 성공률, 결측 구간, 시간표 매칭률

12. 리스크와 대응

리스크	영향	대응
이력이 4주뿐(평일 출퇴근 약 20일)	날씨·사고 같은 날 단위 변화 표본 부족	통계 단위를 거칠게 묶고 발표에서 한계를 먼저 밝힘
하루 1,000건 호출 제한	수집 노선 제한	2호선 30초로 시작, 활용사례 등록 후 확장
API 장애·일시 중단	수집 공백	재시도, 결측 구간 기록, 원본 보존

리스크	영향	대응
열차번호와 시간표 매칭 실패	지연 계산 불가	1주차에 매칭률 먼저 확인
30초 간격 위치 수신	도착 시각 오차, 환승 판정 흔들림	궤적 보간 규칙, 도착정보 API로 샘플 보정
지연이 대부분 0~1분	지연 모델 효과가 작음	EDA 후 환승 성공 확률 모델에 무게
비슷한 서비스 존재	차별성 질문	기존 사례를 먼저 소개, 차별점은 이력 기반 P95·환승 확률·정시율 검증
노선도 UI 작업량	일정 지연	대상 노선만 그리고 일찍 착수

출처

- 서울시 지하철 실시간 열차 위치정보 – 서울 열린데이터광장
- 서울시 지하철 실시간 도착정보(일괄) – 서울 열린데이터광장
- 서울교통공사_열차시간표 – 공공데이터포털
- 서울시 지하철 호선별 역별 시간대별 승하차 인원 정보 – 서울 열린데이터광장
- Open API 이용안내 – 서울 열린데이터광장
- 지하철 혼잡 시간대 – 서울시 교통분야 누리집
- 서울 대중교통 오전 8시 10~19분 가장 혼잡 – 내 손안에 서울
- 열린데이터광장 공지사항 – 서울 열린데이터광장